Digital Signatures

- When user transfers digital coin, they create transaction "my coin → address" and sign with sk
- anyone can verify the signature by knowing: identity + signature.

→ Together with CHFs, digsigs are a fundamental cryptographic building block for cryptocurrencies

→ Digital analog to handwritten sigs on paper.

Components of DigSig.

(1) → key-pair generation function
- sk: secret key
- pk: public key

(2) → Signing function: data/message is signed w/ sk to produce a signature S

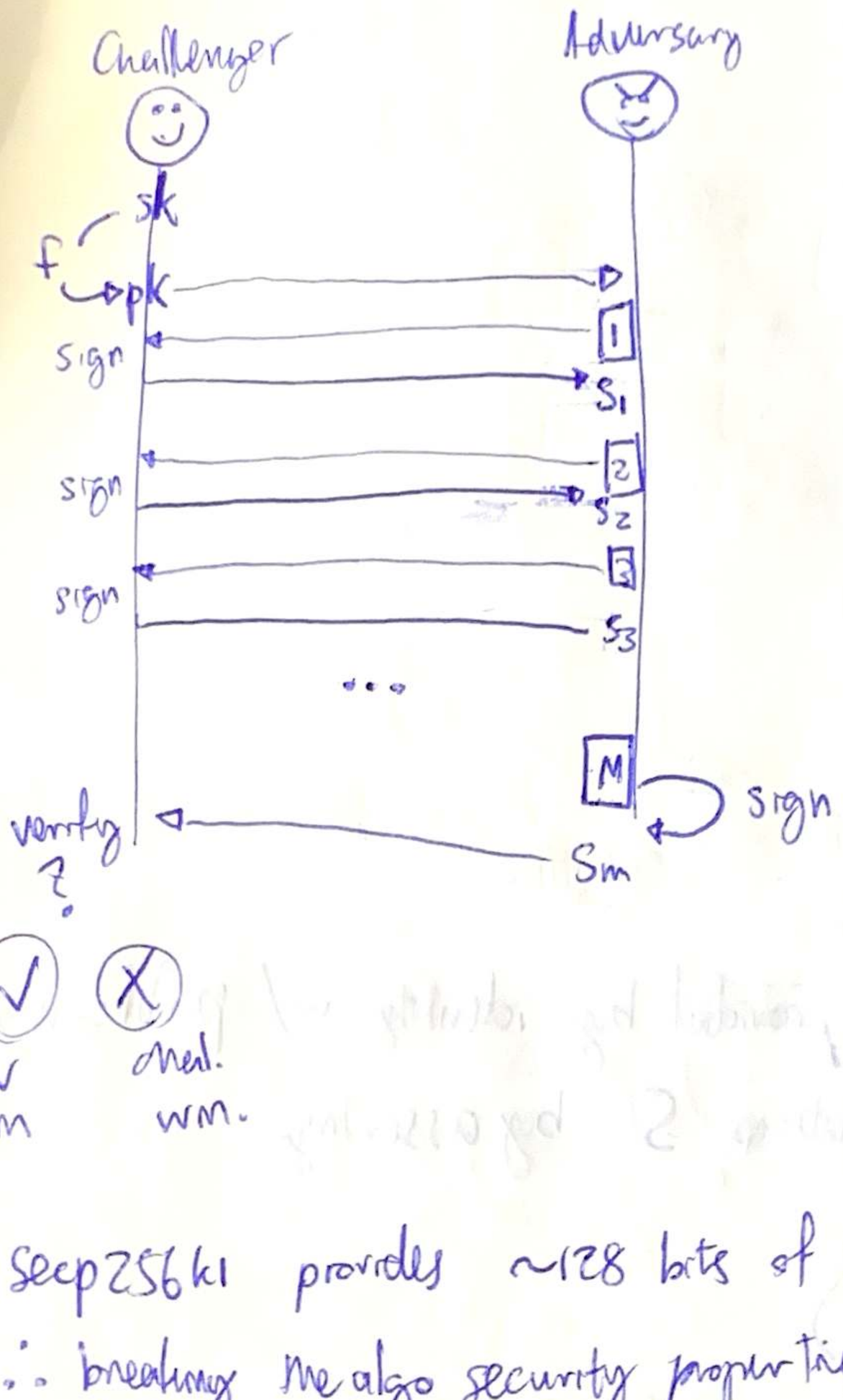
(3) → verification function: S can be verified by any observer as being a product of the secret key that derived the public key that identifies the signer.

* existential unforgeability.
  - a signature is unique to the data it signs.
  - an adversary that has seen numerous signatures of a public key should not be able to forge a new signature for that public key for a new message.

* pk ≡ identity ≡ address
  by signing with the sk of a pk, you speak as the pk

* Bitcoin uses ECDSA (curve: secp256k1) for signing
* Typically the hash ptr of a message is signed rather than the hash itself → signing algos operate on fixed length strings.

Unforgeability game



- secp256k1 provides ~128 bits of security

∴ breaking the algo security property is as computationally difficult as 2^{128} symmetric key crypto ops i.e. hashing.

HIGHLY relevant on source of randomness.

i.e. if you don't have strong randomness, you don't get high min-entropy inputs to hash func.

- decentralised identity management:
  - trad systems: users go to central authority reg. identity
  - now: anyone can run the well known func `generateKeys`

In BTC:
genkeys: sk
pk
hash
address

Digital Signature First Principle:

pk = ~~private~~ public key

sk = secret key

f = hash function (secp256k1)

generate key pair

~~f(k)~~ f(sk) = pk

t = transaction

g = signing function

g(sk, t) = S = signature

→ Given S of transaction t provided by identity w/ public key pk, we verify the signature S by asserting

signature

$$f(S) = g(pk, t)$$

which holds iff same sk was used for S and pk

working.

$$f(S) = g(pk, t)$$

$$\Rightarrow f(g(sk, t)) = g(pk, t)$$

$$\Rightarrow f(g(sk, t)) = g(f(sk), t)$$

which holds if f, g are commutative